Anti-API Documentation

OpenAI-compatible proxy for Antigravity models — works with OpenClaw, Claude Code, Cursor, and any OpenAI-compatible client.

Base URL & Authentication

All requests use this base URL and require an API key:

```
Base URL: https://anti-api-proxy-production.up.railway.app/v1
API Key:  Authorization: Bearer ak-YOUR_API_KEY
```

Available Models

Model ID	Provider	Capabilities
`claude-opus-4-6-thinking`	Claude	Best reasoning thinking
`claude-opus-4-5-thinking`	Claude	Strong reasoning thinking
`claude-sonnet-4-6-thinking`	Claude	Fast reasoning thinking
`claude-sonnet-4-6`	Claude	Fast, latest
`claude-sonnet-4-5-thinking`	Claude	Balanced reasoning thinking
`claude-sonnet-4-5`	Claude	Balanced
`gemini-3-1-pro-high`	Gemini	Latest Gemini, highest quality
`gemini-3-pro-high`	Gemini	High quality
`gemini-3-pro-low`	Gemini	Efficient
`gemini-3-flash`	Gemini	Ultra-fast
`gemini-3-flash-thinking`	Gemini	Fast reasoning thinking

Model ID	Provider	Capabilities
`gpt-oss-120b`	GPT	Open-source, large

🦞 OpenClaw Setup (Step-by-Step)

Follow these steps to connect OpenClaw to Anti-API:

1 **Install OpenClaw** (if not already installed)

```
npm install -g openclaw
```

2 Open your config file

```
nano ~/.openclaw/openclaw.json
```

If the file doesn't exist yet, create it. If it already has content, merge the following into your existing config.

3 Add the Anti-API provider

Paste this into your `openclaw.json`. Replace `ak-YOUR_KEY` with your actual API key:

```
{
  "models": {
    "mode": "merge",
    "providers": {
      "anti-api": {
        "baseUrl": "https://anti-api-proxy-production.up.railway.app/v1",
        "apiKey": "ak-YOUR_KEY",
        "api": "openai-completions",
        "models": [
          { "id": "claude-opus-4-6-thinking", "name": "Claude Opus 4.6 Thinking", "rea
          { "id": "claude-opus-4-5-thinking", "name": "Claude Opus 4.5 Thinking", "rea
          { "id": "claude-sonnet-4-6", "name": "Claude Sonnet 4.6" },
          { "id": "claude-sonnet-4-5", "name": "Claude Sonnet 4.5" },
          { "id": "gemini-3-1-pro-high", "name": "Gemini 3.1 Pro High" },
```

```
          { "id": "gemini-3-flash", "name": "Gemini 3 Flash" }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "anti-api/claude-opus-4-6-thinking"
      }
    }
  }
}
```

4 Restart the gateway

```
openclaw gateway restart
```

5 Verify it works

```
openclaw agent --local -m "Hello, what model are you?" --session-id test --json
```

You should see `"provider": "anti-api"` in the output.

> ✅ **Tools work natively.** OpenClaw's built-in tools (file read/write, web search, process, etc.) work out of the box with all Anti-API models — no extra configuration needed.

🤖 Claude Code Setup (Step-by-Step)

Follow these steps to connect Claude Code to Anti-API:

1 **Install Claude Code** (if not already installed)

```
npm install -g @anthropic-ai/claude-code
```

2 Set environment variables

Add these to your shell config (`~/.zshrc` or `~/.bashrc`):

```
export ANTHROPIC_BASE_URL="https://anti-api-proxy-production.up.railway.app"
export ANTHROPIC_API_KEY="ak-YOUR_KEY"
```

Then reload your shell:

```
source ~/.zshrc
```

3 Set your preferred model

```
claude config set --global model claude-opus-4-6-thinking
```

Or use any model from the list above.

4 Test it

```
claude -p "Hello, what model are you?"
```

You should receive a response from the model.

5 Test with tools

```
claude -p "Read the file package.json and tell me the project name"
```

Claude Code should use the file-read tool and return the project details.

> ✅ **All Claude Code tools work.** File editing, bash commands, web search — everything works through the proxy exactly as with a native Anthropic API key.

> **Note:** Do NOT set `ANTHROPIC_BASE_URL` with a trailing `/v1`. Claude Code adds its own path. The correct value is the root URL without `/v1`.

🔧 Cursor / Continue.dev / Other Tools

Any tool with "OpenAI-compatible" or "Custom OpenAI" support works. Use these settings:

Setting	Value
Base URL	`https://anti-api-proxy-production.up.railway.app/v1`
API Key	`ak-YOUR_KEY`
Model	`claude-opus-4-6-thinking`

API Reference

POST /v1/chat/completions

```
curl https://anti-api-proxy-production.up.railway.app/v1/chat/completions \
  -H "Authorization: Bearer ak-YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "claude-opus-4-6-thinking",
    "messages": [{"role": "user", "content": "Hello!"}],
    "max_tokens": 1024,
    "stream": false
  }'
```

Set `"stream": true` for SSE streaming responses.

GET /v1/models

Returns a list of all available models.

POST /messages

Anthropic Messages API format (used by Claude Code).

Tool Calling

All models support OpenAI-style function/tool calling. The proxy automatically cleans unsupported JSON Schema fields (like `patternProperties`, `$ref`, `additionalProperties`) so your existing tool schemas work without modification.